Berth

Engineering work, releases, and deployment history - docked.

DevSecOps Final Capstone · Session 21

Maverick-sst/Berth · github.com/Maverick-sst/Berth

PROBLEM & DOMAIN

A TaskBoard clone is not a product

- **Generic clones:** rename labels, ship a to-do list, add zero domain depth.
- **Berth's answer:** release readiness and deployment history are first-class product features.
- **Readiness is computed** from real task states - never hard-coded or fabricated.
- **Deployments are auditable:** every attempt recorded with outcome and failure reason.
- **Nothing pretends success** - the UI can't fabricate a passing deployment.

WHAT BERTH IS BUILT WITH

Tech stack

FastAPI
Python 3.12 - REST API

PostgreSQL 16
Alembic migrations

React 18 + Vite
Tailwind v4 / shadcn-style

SQLAlchemy 2
Pydantic v2 validation

Docker Compose
local full stack

GitHub Actions
CI/CD pipeline

Trivy
HIGH/CRITICAL gate

GHCR
immutable SHA tags

Terraform
AWS VPC + EKS

Kubernetes
probes, HPA, ingress

Helm
berth chart

Prometheus + Grafana
observability

REQUEST & DATA FLOW

Architecture

Prometheus scrapes /metrics via ServiceMonitor -> Grafana dashboards (rate, latency, errors)

PRODUCT WALKTHROUGH

What Berth actually does

- **Projects** group tasks; deleting a project with tasks is refused (409).
- **Tasks** status TODO -> IN_PROGRESS -> DONE, priorities, assignee, due dates, search, pagination.
- **Comments & activity** persisted discussion per task; every action writes a real ActivityEvent.
- **Releases** PLANNED -> READY -> DEPLOYING -> DEPLOYED / FAILED, illegal jumps rejected (409).
- **Deployment history** environment, commit SHA, outcome, timestamps, failure reason - failures are first-class.

GITHUB ACTIONS

CI/CD - one authoritative pipeline

test

pytest 53 tests + frontend build

→

build-scan-push

Trivy HIGH/CRIT gate -> GHCR SHA tags

→

smoke-kind

Helm on kind + curl health/ready

- **Security gate is real:** exit-code 1 - findings fail the build before any push.
- **Both images scanned clean** - 0 HIGH/CRITICAL on the published run.
- **Immutable traceability** - ghcr.io/maverick-sst/berth/berth-{backend,frontend}:<commit-sha>.
- **No cloud credentials in CI** - real deploys stay manual via the runbook.

TERRAFORM - AWS

Infrastructure as Code

terraform/ provisions

- VPC 10.20.0.0/16
- 2 public + 2 private subnets
- Single NAT gateway
- EKS 1.31, t3.medium 2-4 node

Cost discipline

- Validated statically: fmt -check + validate pass
- tfvars.example only - no real values committed
- ~$150-220/month while running - documented
- Runbook: plan -> apply (user-approved) -> destroy

HOW BERTH RUNS IN A CLUSTER

Kubernetes + Helm

- **Chart:** helm/taskboard (chart name: berth) - validated with helm lint + helm template.
- **Probes** readiness /ready (checks DB), liveness /health - cheap and meaningful.
- **Scaling** HPA on CPU 60%, 2-6 replicas, resource requests set.
- **Hardening** runAsNonRoot uid 10001/101, capabilities dropped, no privilege escalation.
- **Ingress** /api -> backend :8000, / -> frontend :8080 (nginx class).
- **Smoke-tested in CI** - kind + Helm + real Postgres + curl health/ready/create.

PROMETHEUS + GRAFANA

Observability

- **Instrumentation** prometheus-fastapi-instrumentator exposes /metrics on the backend.
- **Discovery** ServiceMonitor finds the app in any namespace (kube-prometheus-stack values).
- **Dashboard** importable JSON - request rate, p95 latency, error rate, status-code breakdown.
- **Verified, not assumed** - runbook steps to confirm the target is UP and panels are populated.
- **Load demo** scripts/load-test.sh generates traffic so panels come alive on screen.

WHAT KEEPS BERTH HONEST

Quality gates

53 pytest tests
8+ endpoints, isolated in-memory DB

Alembic authoritative
4 migrations, no create_all in prod

Non-root containers
uid 10001 backend / 101 nginx

State machines
release transitions validated 409

409-protected deletes
no orphaned or cascaded data

Token-gated CI reports
hmac.compare_digest bearer

Real activity only
no fabricated feed or metrics

Zero secrets in git
placeholders + runtime injection

WHAT BUILDING BERTH TAUGHT ME

Key learnings

- **Migrations own the schema.** create_all at startup hides drift; Alembic up/down/up on every change.
- **Security gates must fail builds.** Trivy caught 3 HIGH starlette CVEs - fixed by upgrading, not ignoring.
- **Base images go stale.** 41 frontend CVEs traced to frozen packages - apk/apt upgrade at build time.
- **Fabricated data is a smell.** if a dashboard can't be trusted, it isn't a dashboard.
- **Least privilege everywhere.** no cloud creds in CI; GITHUB_TOKEN with packages:write is enough.

Thank you

Berth - engineering work, docked.

github.com/Maverick-sst/Berth · 52 commits · 53 tests · 2 clean images on GHCR